Longest Repeated Substring Hint

💡 Hint

Think about **each character separately**.

Suppose you're trying to make the longest substring of `'a'`.

For every `'a'`, you can look at its positions in the string.

There are two important cases:

1. **A continuous group of the same character**
   - Example: `aaabaaa`
   - Groups of `'a'`: `[aaa] [aaa]`
   - You can swap the `b` with an `a`, joining the two groups.
   - So `3 + 3 = 6`.
2. **Two groups separated by exactly one different character**
   - Example: `a a b a a`
   - You can swap that middle `b` with another `a`.
   - But there's an important limitation: **you cannot create more `'a'` characters than the total number of `'a'`s in the entire string.**

So the key idea is:

> **Find groups of consecutive equal characters and consider joining nearby groups, while never exceeding the total frequency of that character.**

For each character, you'll need:

- the lengths of its consecutive groups
- the total frequency of that character

Then find the maximum possible answer.

Think about how you would represent the consecutive groups — e.g. for `"aaabaaaacaa"`:

```
aaa | b | aaaa | c | aa
 3      4          2
```

For `'a'`, which groups can potentially be combined, and what should the answer be limited by? That's the core of the problem.

Absolutely. Let's start **from zero** and build the idea slowly. The confusing part of this problem is understanding **what the one swap can actually accomplish**.

---

1. What are we trying to do?

We have a string, and we are allowed to **swap any two characters once**.

We want:

> After the swap, what is the maximum length of a substring containing only the **same character**?

For example:

```
aaabaaa
```

Currently:

```
aaa b aaa
```

The longest repeated substring is `aaa` → length `3`.

But we can swap:

```
aaa b aaa
    ↑
```

the `b` with an `a` somewhere else.

For example:

```
aaabaaa
   ↓
aaaaaaa
```

Actually, because there are only **six `a`s**, the result becomes:

```
aaaaaa
```

So answer = `6`.

---

2. First important observation

Forget swapping for a moment.

Look at the string as **groups of consecutive same characters**.

For:

```
aaabaaa
```

we have:

```
aaa | b | aaa
```

So for character `'a'`:

```
Group 1 = 3 a's
Group 2 = 3 a's
```

And total number of `a`s is:

```
6
```

3. What can one swap do?

Suppose we have:

```
aaa b aaa
```

There is exactly **one different character** between the two groups:

```
aaa | b | aaa
```

We can take an `'a'` from somewhere else and swap it with `b`.

Then:

```
aaa b aaa
    ↓
aaa a aaa
```

Now:

```
aaaaaaa
```

But wait!

There were only **6 a's** originally.

We cannot magically create a 7th `a`.

So the maximum is:

```
min(3 + 3 + 1, total_a)
= min(7, 6)
= 6
```

This `min()` is **very important**.

---

4. Let's understand another example

Take:

```
aaabaa
```

Groups:

```
aaa | b | aa
```

Lengths:

```
3 + 2
```

There are total:

```
5 a's
```

We can replace `b` with another `a`.

So:

```
aaa b aa
```

becomes:

```
aaa a aa
```

giving:

```
aaaaaa
```

But there are only 5 `a`s!

So the actual answer is:

```
5
```

Mathematically:

```
min(3 + 2 + 1, 5)
= min(6, 5)
= 5
```

5. Now what about this?

```
ababa
```

There are three `a`s:

```
a | b | a | b | a
```

Can we make:

```
aaa
```

Yes.

For example, swap the first `b` with the last `a`:

```
a b a b a
↓
a a a b b
```

Now we have:

```
aaa
```

So answer = `3`.

But notice something:

We **cannot** make `aaaa`, because there are only **3 a's total**.

Therefore:

```
answer <= total number of that character
```

6. So what exactly should we look for?

For every character, say `'a'`:

Step 1

Find all consecutive groups of `'a'`.

Example:

```
aaabaaaacaa
```

Break it:

```
aaa | b | aaaa | c | aa
```

So:

```
3, 4, 2
```

Step 2

Look at groups that have **one character between them**.

Here:

```
aaa | b | aaaa
```

The two `a` groups can be joined using one swap.

Their combined length would be:

```
3 + 4 + 1 = 8
```

But total number of `a`s is:

```
3 + 4 + 2 = 9
```

So:

```
min(8, 9) = 8
```

Another pair:

```
aaaa | c | aa
```

would give:

```
4 + 2 + 1 = 7
```

So currently best = `8`.

---

7. But there's another case!

Suppose:

```
aaaaabc
```

Groups:

```
aaaaa | b | c
```

We don't really need to join two groups.

We can simply take the existing group:

```
aaaaa
```

and potentially swap another `'a'` from elsewhere into it.

So if a group has length `5` and there are `7` total `'a'` characters:

```
aaaaa ... a ...
```

we could make the group:

```
aaaaaa
```

or whatever the total count allows.

Therefore, we also need to consider:

> **The longest existing group + 1**, limited by the total count.

---

8. The entire idea in one picture

For each character:

```
          ┌── consecutive group
          ↓
aaa | b | aaaa | c | aa
 ↑              ↑
 group 1        group 2
```

If two groups are separated by exactly **one different character**:

```
aaa | b | aaaa
```

we can potentially join them:

```
3 + 1 + 4
```

But:

```
result <= total number of 'a'
```

So:

```
candidate = min(group1 + group2 + 1, totalCount)
```

And separately:

```
candidate = min(longestGroup + 1, totalCount)
```

Take the maximum candidate.

---

9. The key thing to understand

You **don't need to actually perform the swap**.

You just need to ask:

> "If I choose this character, which consecutive groups can I connect with my one swap?"

That's what makes the problem manageable.

Since the string contains only lowercase letters, you can process the groups for each character and keep the best answer.

If you want, **next I can show you how to convert this exact idea into code step-by-step**, starting with how to find the consecutive groups.

Sure. Let's implement the exact idea we discussed.

C++ Solution

```
C++
class Solution {
public:
    int maxRepOpt1(string text) {

        int n = text.size();
        int total[26] = {0};

        // Count total occurrences of every character
        for (char c : text) {
            total[c - 'a']++;
        }

        int ans = 0;

        // Process consecutive groups
        for (int i = 0; i < n; ) {

            int j = i;

            // Find length of current group
            while (j < n && text[j] == text[i]) {
                j++;
            }

            int len1 = j - i;
            int idx = text[i] - 'a';

            // Case 1:
            // Take this group and bring one same character from outside
            ans = max(ans, min(len1 + 1, total[idx]));

            // Case 2:
            // Check if another same-character group exists
```

```
            // after exactly one different character
            if (j + 1 < n && text[j + 1] == text[i]) {

                int k = j + 1;

                // Find length of second group
                while (k < n && text[k] == text[i]) {
                    k++;
                }

                int len2 = k - (j + 1);

                // Join the two groups using one swap
                ans = max(ans, min(len1 + len2 + 1, total[idx]));
            }

            i = j;
        }

        return ans;
    }
};
```

Let's dry-run the important part

For:

```
aaabaaa
```

Groups:

```
aaa | b | aaa
```

For the first `aaa`:

```
len1 = 3
```

The code sees:

```
C++
text[j + 1] == text[i]
```

because after `b`, there is another `a` group.

So:

```
len1 = 3
len2 = 3
total a = 6
```

Candidate:

```
C++
min(3 + 3 + 1, 6)
= min(7, 6)
= 6
```

Therefore:

```
answer = 6
```

Complexity

```
Time:  O(n)
Space: O(1)
```

The important part to understand in the code is this:

```
C++
min(len1 + len2 + 1, total[idx])
```

`+1` → the different character between the two groups can become the same character.

`total[idx]` → we cannot have more copies of a character than actually exist in the original string.

This line:

```
C++
int idx = text[i] - 'a';
```

is just converting the **character into an array index**.

We have:

```cpp
int total[26] = {0};
```

This array stores the count of each lowercase character:

```
index:  0  1  2  3  ... 25
char:   a  b  c  d  ...  z
```

Because characters have numeric ASCII values:

```
'a' = 97
'b' = 98
'c' = 99
...
```

So:

```cpp
text[i] - 'a'
```

gives:

```
'a' - 'a' = 0
'b' - 'a' = 1
'c' - 'a' = 2
'd' - 'a' = 3
...
'z' - 'a' = 25
```

Example

Suppose:

```cpp
text[i] = 'c';
```

Then:

```cpp
int idx = text[i] - 'a';
```

becomes:

```cpp
idx = 'c' - 'a';
idx = 99 - 97;
idx = 2;
```

So:

```cpp
total[idx]
```

means:

```cpp
total[2]
```

which stores the **total number of `'c'` characters**.

For example, if:

```
text = "aaabaaaac"
```

then:

```
total[0] = 7   // a
total[1] = 0   // b
total[2] = 1   // c
```

So when we're currently processing `'c'`:

```cpp
int idx = 'c' - 'a';  // 2
```

and:

```C++
total[idx]
```

gives:

```C++
total[2]  // total number of c's
```

In short

```C++
int idx = text[i] - 'a';
```

means:

> **"Convert this character (`a-z`) into a number (`0-25`) so I can use it as an index in `total[26]`."**